Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2

Технології розроблення програмного забезпечення

Тема: «Особиста бухгалтерія»

Виконала:

Студентка групи ІА-34

Дригант А.С

Перевірив:

Мягкий Михайло Юрійович

Київ-2025

Тема: Основи проектування

Мета: Обрати зручну систему побудови UML-діаграм та навчитися будувати діаграми варіантів використання для системи що проєктується, розробляти сценарії варіантів використання та будувати діаграми класів предметної області.

2.1. Теоретичні відомості

2.1.1 Вступ до мови UML

Мова UML є загальноцільовою мовою візуального моделювання, яка розроблена для специфікації, візуалізації, проєктування та документування компонентів програмного забезпечення, бізнес-процесів та інших систем [3]. Мова UML є досить строгим та потужним засобом моделювання, який може бути ефективно використаний для побудови концептуальних, логічних та графічних моделей складних систем різного цільового призначення. Ця мова увібрала в себе найкращі якості та досвід методів програмної інженерії, які з успіхом використовувалися протягом останніх років при моделюванні великих та складних систем.

З погляду методології ООАП (об'єктно-орієнтованого аналізу та проєктування) досить повна модель складної системи є певною кількістю взаємопов'язаних уявлень (views), кожне з яких відображає аспект поведінки або структури системи. У цьому найзагальнішими уявленнями складної системи прийнято вважати статичне і динамічне, які у своє чергу можуть поділятися інші більш приватні.

Принцип ієрархічної побудови моделей складних систем передбачає розгляд процес побудови моделей на різних рівнях абстрагування або деталізації в рамках фіксованих уявлень.

Рівень представлення (layer) – спосіб організації та розгляду моделі на одному рівні абстракції, що представляє горизонтальний зріз архітектури моделі, тоді як розбиття представляє її вертикальний зріз.

При цьому вихідна або початкова модель складної системи має найбільш загальне уявлення та відноситься до концептуального рівня. Така модель, що

отримала назву концептуальної, будується на початковому етапі проєктування і може не містити багатьох деталей та аспектів системи, що моделюється. Наступні моделі конкретизують концептуальну модель, доповнюючи її уявленнями логічного та фізичного рівня.

Загалом процес ООАП можна розглядати як послідовний перехід від розробки найбільш загальних моделей та уявлень концептуального рівня до більш приватних і детальних уявлень логічного та фізичного рівня. У цьому кожному етапі ООАП дані моделі послідовно доповнюються дедалі більше деталей, що дозволяє їм адекватно відбивати різні аспекти конкретної реалізації складної системи.

В рамках мови UML уявлення про модель складної системи фіксуються у вигляді спеціальних графічних конструкцій, що отримали назву діаграм.

Діаграма (diagram) – графічне уявлення сукупності елементів моделі у формі зв'язкового графа, вершинам і ребрам (дугам) якого приписується певна семантика. Нотація канонічних діаграм є основним засобом розробки моделей мовою UML.

У нотації мови UML визначено такі види діаграм:

- варіантів використання (use case diagram);
- класів (class diagram);
- кооперації (collaboration diagram);
- послідовності (sequence diagram);
- станів (statechart diagram);
- діяльності (activity diagram);
- компонентів (component diagram);
- розгортання (deployment diagram).

Перелічені діаграми є невід'ємною частиною графічної нотації мови UML. Понад те, процес ООАП нерозривно пов'язаний з процесом побудови цих діаграм. При цьому сукупність побудованих таким чином діаграм є самодостатньою в тому сенсі, що в них міститься вся інформація, яка потрібна для реалізації проєкту складної системи.

Кожна з цих діаграм деталізує та конкретизує різні уявлення про модель складної системи у термінах мови UML. При цьому діаграма варіантів використання являє собою найбільш загальну концептуальну модель складної системи, яка є вихідною для побудови інших діаграм. Діаграма класів, за своєю суттю, логічна модель, що відбиває статичні аспекти структурної побудови складної системи.

Діаграми кооперації та послідовностей є різновидами логічної моделі, які відображають динамічні аспекти функціонування складної системи. Діаграми станів та діяльності призначені для моделювання поведінки системи. І, нарешті, діаграми компонентів і розгортання служать уявлення фізичних компонентів складної системи і тому представляють її фізичну модель.

2.2.2. Діаграма варіантів використання (Use-Cases Diagram)

Діаграма варіантів використання (Use-Cases Diagram) – це UML діаграма за допомогою якої у графічному вигляді можна зобразити вимоги до системи, що розробляється [3]. Діаграма варіантів використання – це вихідна концептуальна модель проєктованої системи, вона не описує внутрішню побудову системи.

Діаграми варіантів використання призначені для:

- визначення загальної межі функціональності проєктованої системи;
- формулювання загальних вимоги до функціональної поведінки проєктованої системи;
- подальшої розробка вихідної концептуальної моделі системи (діаграми класів);
- створення основи для виконання аналізу, проєктування, розробки та тестування.

Діаграми варіантів використання є відправною точкою при збиранні вимог до програмного продукту та його реалізації. Дана модель будується на аналітичному етапі побудови програмного продукту (збір та аналіз вимог) і дозволяє бізнес-аналітикам отримати більш повне уявлення про необхідне програмне забезпечення та документувати його.

Діаграма варіантів використання складається з низки елементів. Основними елементами є: варіанти використання або прецеденти (use case), актор або дійова особа (actor) та відносини між акторами та варіантами використання (relationship).

2.2.3. Актори (actor)

Актором називається будь-який об'єкт, суб'єкт чи система, що взаємодіє з модельованою бізнес-системою ззовні для досягнення своїх цілей або вирішення певних завдань [3]. Це може бути людина, технічний пристрій, програма або будь-яка інша система, яка служить джерелом впливу на систему, що моделюється. Актора можна розглядати як роль, яка виконується людиною в системі. Зображення акторів на діаграмах представлено на рисунку 2.1.

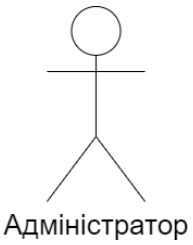


Рисунок 2.1. Зображення акторів на діаграмах UML

Ім'я актора має бути достатньо інформативним з точки зору програмного забезпечення, що розробляється, або предметної області. Для цього підходять найменування посад у компанії (наприклад, продавець, касир, менеджер, президент).

2.2.4. Варіанти використання (use case)

Варіант використання служить для опису служб, які система надає актору. Інакше кажучи кожен варіант використання визначає набір дій, здійснюваний системою під час діалогу з актором. Кожен варіант використання являє собою послідовність дій, який повинен бути виконаний системою, що проєктується при взаємодії її з відповідним актором, самі ці дії не відображаються на діаграмі.

Варіант використання відображається еліпсом, всередині якого міститься його коротке ім'я з великої літери у формі іменника або дієслова. Приклад зображення варіанта використання на діаграмі представлено на рисунку 2.2.

Рисунок 2.2. Позначення варіанта використання на діаграмах

При проєктуванні варіантів використання, потрібно приймати до уваги, що з назви варіанта використання має бути зрозуміла тривалість та результат його виконання.

Приклади варіантів використання: реєстрація, авторизація, оформлення замовлення, переглянути замовлення, перевірка стану поточного рахунку тощо.

2.2.5 Відносини на діаграмі варіантів використання

Відношення (relationship) – семантичний зв'язок між окремими елементами моделі.

Один актор може взаємодіяти з кількома варіантами використання. У цьому випадку цей актор звертається до кількох служб цієї системи. У свою чергу, один варіант використання може ініціюватися декількома акторами, надаючи для них свій функціонал.

Існують такі відносини: асоціації, узагальнення, залежність (складається з включення та розширення).

Асоціація (association) – узагальнене, невідоме ставлення між актором та варіантом використання.

Позначається суцільною лінією між актором та варіантом використання. Розрізняють ненаправлену (двонаправлену) асоціацію та однонаправлену асоціацію. Ненаправлена асоціація показує взаємодію без акцента на напрямок, або коли напрямок ще не аналізувався. Відображення ненаправленої асоціації між актором «Покупець» та варіантом використання «Перегляд списку

доступних товарів» представлено на рисунку 2.3.

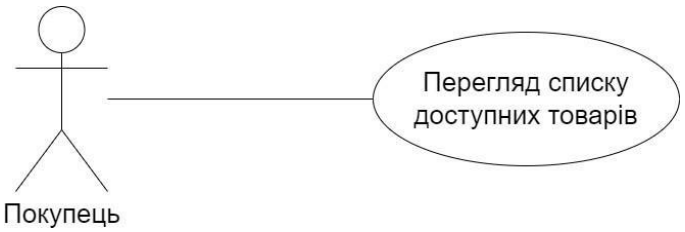


Рисунок 2.3. Ненаправлена асоціація

Спрямована, або направлена асоціація (directed association) – також показує що актор асоціюється з варіантом використання але показує, що варіант використання ініціалізується актором. Позначається стрілкою. Приклад направлених асоціацій представлено на рисунку 2.4.

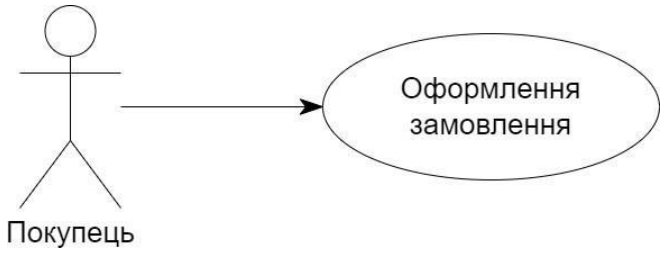


Рисунок 2.4. Приклад спрямованої асоціації

Спрямована асоціація дозволяє запровадити поняття основного актора (він є ініціатором асоціації) та другорядного актора (варіант використання є ініціатором, тобто передає акторові довідкові відомості або звіт про виконану роботу). Приклад асоціації направленої до актора представлено на рисунку 2.5.

Рисунок 2.5. Приклад асоціації направленої до актора

Відношення узагальнення (generalization) – показує, що нащадок

успадковує атрибути у свого прямого батьківського елементу. Тобто, один елемент моделі є спеціальним або окремим випадком іншого елемента моделі. Може застосовуватися як до акторів, так і до варіантів використання.

Графічно відношення узагальнення позначається суцільною лінією зі стрілкою у формі незафарбованого трикутника, яка вказує на батьківський варіант використання.

На рисунку 2.6 показано відношення узагальнення. Варіант використання «Оплата замовлення» називається предком (чи батьком), а варіант використання «Оплата замовлення банківською картою» – нащадком (чи дочірнім) стосовно «Оплата замовлення».

Рисунок 2.6. Відношення узагальнення

Особливості використання відношення узагальнення:

- головною особливістю відношення узагальнення є те, що воно може пов'язувати між собою лише елементи одного типу;
- один варіант використання може мати кілька батьківських варіантів використання (множинне успадкування);
- один варіант використання може бути предком кількох дочірніх варіантів використання (таксономічний характер відносин).

Також існує узагальнення між акторами, яке представлено на рисунку 2.7.

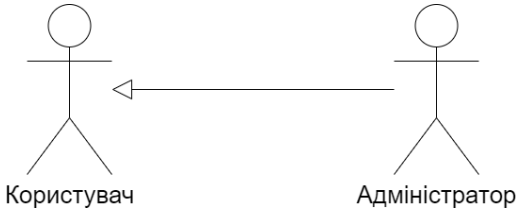


Рисунок 2.7. Відношення узагальнення між акторами

У наведеному прикладі Адміністратор успадковує всі атрибути свого предка Користувача, але може мати свої індивідуальні, які не зображені на рисунку.

Відношення залежності (dependency) визначається як форма взаємозв'язку між двома елементами моделі, призначена для специфікації тієї обставини, що зміна одного елемента моделі призводить до зміни деякого іншого елемента. Загалом залежність є спрямованим бінарним ставленням, яке пов'язує між собою два елементи моделі: незалежний та залежний.

Відношення включення (include) – окремий випадок загального відношення залежності між двома варіантами використання, при якому деякий варіант використання містить поведінку, визначену в іншому варіанті використання.

Графічне зображення – пунктирна стрілка з стереотипом << include>>.

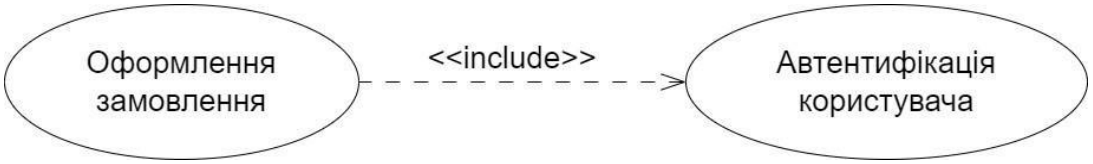


Рисунок 2.8. Відношення включення

Залежний варіант використання «Автентифікація користувача» називають базовим, а незалежний – включається («Оформлення замовлення»). На рисунку включення означає, що кожне виконання варіанта використання «Оформлення замовлення» завжди включатиме виконання варіанта використання «Автентифікація користувача». На практиці відношення включення використовується для моделювання ситуації, коли існує загальна частина поведінки двох або більше варіантів використання. Загальна частина виноситься на окремий варіант використання.

Особливості використання відношення включення:

- один базовий варіант використання може бути пов'язаний ставленням включення з кількома варіантами використання, що включаються;

- один варіант використання може бути включений до інших варіантів використання;
- в одній діаграмі варіантів використання не може бути замкнутого шляху стосовно включення.

Відношення розширення (extend) – показує, що варіант використання розширює базову послідовність дій та вставляє власну послідовність. У цьому на відміну типу відносин «включення» розширена послідовність може здійснюватися залежно від певних умов.

Графічне зображення відношення розширення – пунктирна стрілка спрямована від залежного варіанта (розширює) до незалежного варіанта (базового) з ключовим словом <<extend>>.

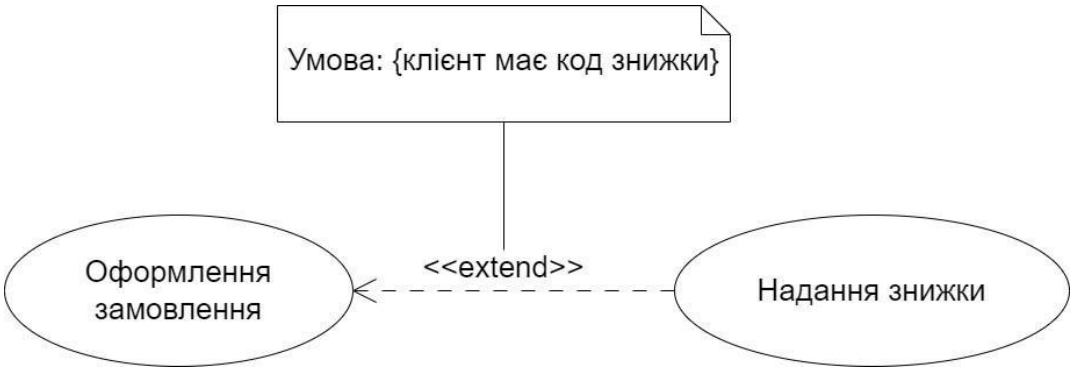


Рисунок 2.9. Приклад відношення розширення

Варіант використання «Оформлення замовлення» є базовим і може бути розширений варіантом використання "Надання знижки", наприклад, за наявності у покупця коду на знижку.

Відношення розширення дозволяють моделювати той факт, що базовий варіант використання може приєднувати до своєї поведінки деякі додаткові поведінки за рахунок розширення у варіанті іншому варіанті використання

Наявність такого відношення передбачає перевірку умови у точці розширення (extension point) у базовому варіанті використання. Точка розширення може мати деяке ім'я і зображена за допомогою примітки.

Відношення розширення з точкою розширення Особливості використання

відношення розширення:

- один базовий варіант використання може мати кілька точок розширення, з кожною з яких повинен бути пов'язаний варіант використання, що розширює;
- один розширюючий варіант використання може бути пов'язаний з відношенням розширення з декількома базовими варіантами використання;
- розширюючий варіант використання може бути пов'язаний з відношенням розширення з декількома базовими варіантами використання;
- розширюючий варіант використання може, своєю чергою, мати власні розширюючі варіанти використання;
- на одній діаграмі варіантів використання не може бути замкнутого шляху щодо розширення.

2.2.6. Сценарії використання

Діаграма варіантів використання надає знання про необхідну функціональність системи в інтуїтивно-зрозумілому вигляді, проте не несе відомостей про фактичний спосіб її реалізації. Конкретні варіанти використання можуть звучати надто загально та не бути придатними для реалізації програмістами.

Для документації варіантів використання у вигляді певної специфікації та усунення неточностей і непорозуміння діаграм варіантів використання, як частину процесу збору та аналізу вимог складаються звані сценарії використання.

Сценарії використання – це текстові уявлення тих процесів, які відбуваються при взаємодії користувачів системи та самої системи. Вони є чітко формалізованими, покроковими інструкціями, що описують той чи інший процес у термінах кроків досягнення мети. Сценарії використання однозначно визначають кінцевий результат.

Сценарії використання описують варіанти використання природною мовою. Вони мають загального, шаблонного вигляду написання, проте рекомендується такий перелік для опису:

- Передумови – умови, які повинні бути виконані для виконання даного варіанту використання;
- Постумови– що виходить в результаті виконання даного варіанту використання;
- Взаємодіючі сторони;
- Короткий опис;
- Основний перебіг подій;
- Винятки;
- Примітки.

Приклад

Розглянемо приклад входу студента до електронного кампуса (варіант використання «отримання електронних матеріалів з кампуса», один із підпроцесів).

Передумови: Немає.

Постумови: У разі успішного виконання, користувач входить до системи. У протилежному випадку стан системи не змінюється.

Взаємодіючі сторони: Студент, Кампус.

Короткий опис: Цей варіант використання визначає вхід користувача до системи реєстрації курсів.

Основний потік подій.

Цей варіант використання починає виконуватися, коли користувач хоче увійти до системи реєстрації курсів.

1. Система запитує ім'я користувача та пароль.

2. Користувач вводить ім'я та пароль.

3. Система перевіряє ім'я та пароль, після чого відкривається доступ до системи. Якщо ім'я та пароль неправильні, Виняток №1.

Винятки

Виняток №1: Неправильне ім'я/пароль. Якщо під час виконання Основного потоку виявиться, що користувач ввів неправильне ім'я та пароль, система виводить повідомлення про помилку. Користувач може повернутися до початку Основного потоку або відмовитись від входу в систему, при цьому виконання варіанта використання завершується.

Примітки Відсутні.

2.2.7. Діаграми класів

Діаграми класів використовуються при моделюванні програмних систем найчастіше. Вони є однією із форм статичного опису системи з погляду її проєктування, показуючи її структуру [3]. Діаграма класів не відображає динамічної поведінки об'єктів зображених на ній класів. На діаграмах класів показуються класи, інтерфейси та відносини між ними.

Представлення класів

Клас – це основний будівельний блок програмної системи. Це поняття є і в мовах програмування, тобто між класами UML та програмними класами є відповідність, що є основою для автоматичної генерації програмних кодів або для виконання реінжинірингу. Кожен клас має назву, атрибути та операції. Клас на діаграмі показується як прямокутник, розділений на 3 області. У верхній міститься назва класу, у середній – опис атрибутів (властивостей), у нижній – назви операцій – послуг, що надаються об'єктами цього класу.

Атрибути класу визначають склад та структуру даних, що зберігаються в об'єктах цього класу. Кожен атрибут має ім'я та тип, який визначає, які дані він представляє.

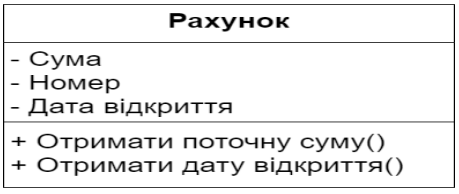


Рисунок 2.10. Зображення класу в нотації UML

При реалізації об'єкта в програмному коді для атрибутів буде виділено пам'ять, необхідна зберігання всіх атрибутів, і кожен атрибут матиме конкретне значення у час роботи програми. Об'єктів одного класу у програмі може бути скільки завгодно багато, всі вони мають однаковий набір атрибутів, описаний у класі, але значення атрибутів у кожного об'єкта свої і можуть змінюватися в ході виконання програми.

Для кожного атрибуту класу можна встановити видимість (visibility). Ця характеристика показує, чи доступний атрибут для інших класів. У UML визначено такі рівні видимості атрибутів:

• + Відкритий (public) – атрибут видно для будь-якого іншого класу (об'єкта);

• ~ В межах пакету (package) – атрибут видно для будь-якого іншого класу, який знаходиться в цьому ж пакеті;

• # Захищений (protected) – атрибут видно для нащадків цього класу;

• - Закритий (private) – атрибут не видно зовнішніми класами (об'єктами) і може використовуватися лише об'єктом, що його містить.

Клас містить оголошення операцій, що є визначення запитів, які повинні виконувати об'єкти даного класу. Кожна операція має сигнатуру, що містить ім'я операції, тип значення, що повертається, і список параметрів, який може бути порожнім. Реалізація операції як процедури – це спосіб, що належить класу. Для операцій, як й у атрибутів класу, визначено поняття «видимість». Закриті операції є внутрішніми для об'єктів класу і недоступні з інших об'єктів. Інші утворюють інтерфейсну частину класу і є засобом інтеграції класу до програмної системи.

Відносини між класами

На діаграмах класів зазвичай показуються асоціації та узагальнення.

Кожна асоціація несе інформацію про зв'язки між об'єктами усередині програмної системи. Найчастіше використовуються бінарні асоціації, які пов'язують два класи. Асоціація може мати назву, яка має виражати суть

відображуваного зв'язку. Крім назви, асоціація може мати таку характеристику як множинність. Вона показує, скільки об'єктів кожного класу може брати участь у асоціації.

Множинність вказується у кожного кінця асоціації (полюса) і задається конкретним числом чи діапазоном чисел. Множинність, зазначена у вигляді зірочки, передбачає будь-яку кількість (у тому числі й нуль). Пов'язані між собою можуть і об'єкти одного класу, тому асоціація може пов'язувати клас із собою. Наприклад, для класу «Мешканець міста» можна ввести асоціацію «Сусідство», яке дозволить знаходити всіх сусідів конкретного мешканця.

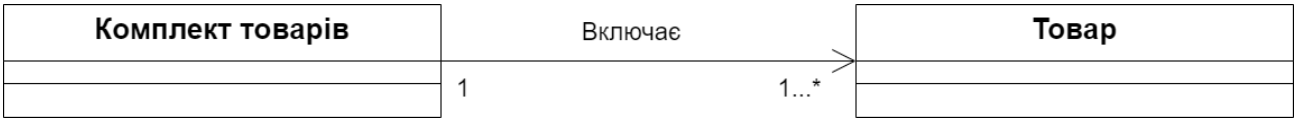


Рисунок 2.11. Асоціація між класами

Асоціація «включає» (рисунок 2.11) показує, що комплект може містити кілька різних товарів. В даному випадку спрямована асоціація дозволяє знайти всі види товарів, що входять у комплект, але не дає відповіді на запитання, чи товар цього виду входить у який-небудь набір.

Асоціація сама може мати властивості класу, тобто мати атрибути та операції. У цьому випадку вона називається клас-асоціацією і може розглядатися як клас, у якого крім явно зазначених атрибутів та операцій є посилання на обидва зв'язувані нею класи.

Асоціація – найбільш загальний вид зв'язку між двома класами системи. Як правило, вона відображає використання одного класу іншим за допомогою певної якості або поля.

Узагальнення (успадкування) на діаграмах класів використовується, щоб показати зв'язок між класом-батьком та класом-нащадком. Воно вводиться на діаграму, коли виникає різновид будь-якого класу, і навіть у випадках, як у системі виявляються кілька класів, які мають подібну поведінку (у разі загальні

елементи поведінки виносяться на більш високий рівень, утворюючи узагальнюючий клас).

Агрегацією позначається відношення частина-ціле, коли об'єкти одного класу входять до об'єкта іншого класу. Типовим прикладом таких відносин є списки об'єктів. У цьому випадку список буде виступати агрегатом, а об'єкти, що входять до списку, елементами, що агрегуються.

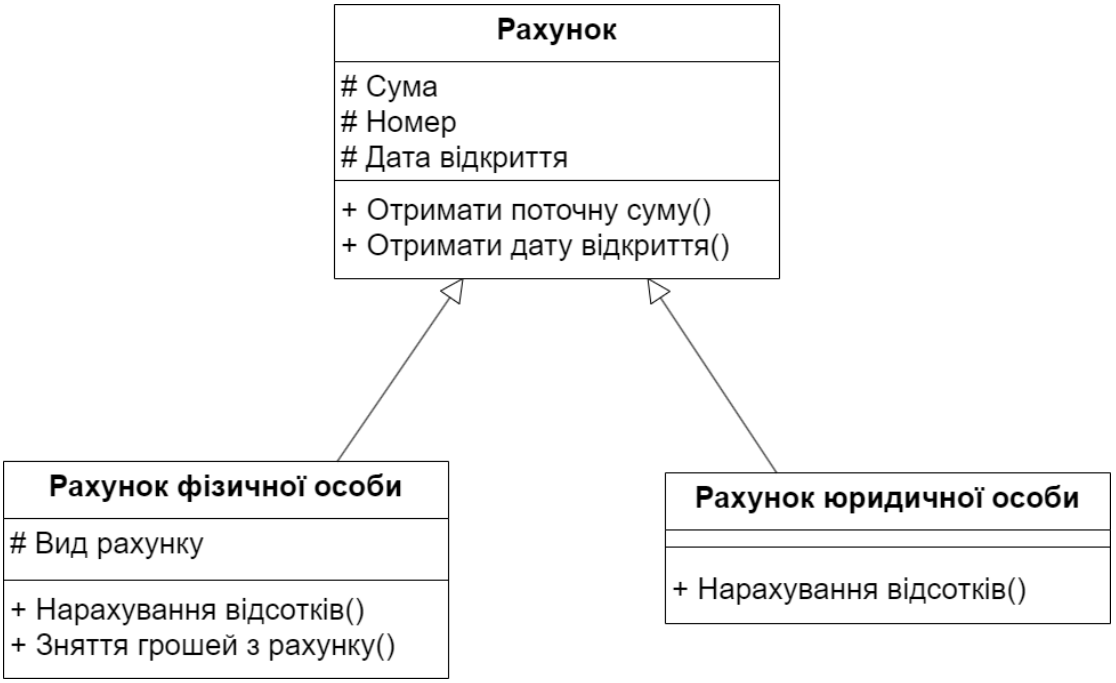


Рисунок 2.12. Приклад наслідування

Приклад агрегації показано на рисунку 2.13. Ромб в нотації частина ціле ставиться біля класу, що представляє ціле у цьому відношенні. Навчальна група містить список студентів. Композиція – це є відображенням зв’язку, при якому об’єкти «Навчальна група» та «Студент» можуть існувати один без одного. Наприклад, при видалені об’єкту «Навчальна група» об’єкти «Студент» пов’язані з нею продовжують існувати.

Композицією також позначається відношення частина-ціле, але позначає тісніший зв'язок між елементами, що представляють ціле та частини.

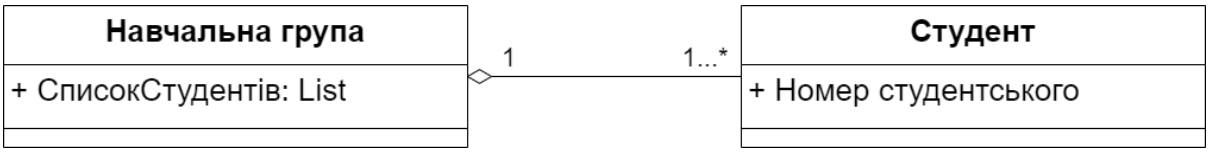


Рисунок 2.13. Приклад використання агрегації

Прикладом композиції можна назвати зв’язко між будівлею і приміщеннями в будівлі. Будівля та приміщення складають неділиме ціле. Приклад композиції показано на рисунку 2.14. Композиція показує, що квартири не можуть існувати за межами житлового будинку і перенести квартири до іншого житлового будинку неможливо.

Рисунок 2.14. Приклад композиції

Якщо говорити про відображення агрегації та композиції в програмному коді, то агрегація, як правило відображується як посилання на об’єкт, а композиція – це змінна типу структури. Біля цілого може бути вказана назва, яка вказує на назву поля класу, яке відображений на діаграмі цим відношенням.

Застосування діаграм класів

Діаграми класів створюються при логічному моделюванні програмних систем та служать для наступних цілей:

• Для моделювання даних. Аналіз предметної області дозволяє виявити основні характерні нею сутності та зв'язку з-поміж них. Це зручно моделюється з допомогою діаграм класів. Ці діаграми є основою побудови концептуальної моделі.

• Для представлення архітектури програмної системи. Можна виділити архітектурно значимі класи і відобразити їх на діаграмах, що описують архітектуру програмної системи.

• Для моделювання навігації екранів. На таких діаграмах показуються прикордонні класи та їхній логічний взаємозв'язок. Інформаційні поля моделюються як атрибути класів, а кнопки, що управляють, – як операції та відносини.

• Для моделювання логіки програмних компонентів.

• Для моделювання логіки обробки даних.

2.2.8. Логічна структура бази даних

Розрізняють дві моделі бази даних – логічну та фізичну. Фізична модель бази даних представляє собор набір бінарних даних у вигляді файлів, структурованих та згрупованих згідно з призначенням (сегменти, екстенти та ін.), що використовується для швидкого та ефективного отримання інформації з жорсткого диска, а також для компактного зберігання та розміщення даних на жорсткому диску.

Логічна модель бази даних є структурою таблиць, уявлень, індексів та інших логічних елементів бази даних, що дозволяють власне програмування та використання бази даних.

Процес створення логічної моделі бази даних зветься проєктування бази даних (database design). Проєктування відбувається у зв'язку з опрацюванням архітектури програмної системи, оскільки база даних створюється зберігання даних, одержуваних з програмних класів.

Відповідно можна розрізнити кілька підходів до зв'язування програмних класів та таблиць: одна таблиця – один клас, одна таблиця – кілька класів, один клас – кілька таблиць. Залежно від обраного підходу, буде ускладнюватись (і уповільнюватися) робота з базою даних при додаванні даних, або отримання даних з БД та парсинг їх у відповідні класи.

З іншого боку, якщо програмні класи є сутності проєктованої системи (елементи предметної області), то таблиці відображають їх технічну реалізацію та спосіб зберігання та зв'язку.

Основним керівництвом під час проєктування таблиць є т. зв. нормальні форми баз даних.

Нормальні форми

Нормальна форма – властивість відношення в реляційній моделі даних, що характеризує його з погляду надмірності, що потенційно призводить до логічно помилкових результатів вибірки або зміни даних. Нормальна форма окреслюється сукупністю вимог, яким має задовольняти ставлення.

Нормалізація призначена для приведення структури БД до виду, що забезпечує мінімальну логічну надмірність, і не має на меті зменшення або збільшення продуктивності роботи або зменшення або збільшення фізичного обсягу бази даних. Кінцевою метою нормалізації є зменшення потенційної суперечливості інформації, що зберігається в базі даних [4].

Змінна відношення знаходиться в першій нормальній формі (1НФ) тоді і тільки тоді, коли в будь-якому допустимому значенні відношення кожен його кортеж містить лише одне значення для кожного з атрибутів.

Змінна відношення знаходиться в другій нормальній формі тоді і тільки тоді, коли вона знаходиться в першій нормальній формі, і кожен неключовий атрибут функціонально повно залежить від її потенційного ключа.

Змінна відношення знаходиться у третій нормальній формі тоді і лише тоді, коли вона знаходиться у другій нормальній формі, і відсутні транзитивні функціональні залежності неключових атрибутів від ключових.

Змінна відношення знаходиться в нормальній формі Бойса-Кодда (інакше – в посиленій третій нормальній формі) тоді і тільки тоді, коли кожна її нетривіальна і неприведена зліва функціональна залежність має в якості свого детермінанта певний потенційний ключ.

2.2.9. Проєктування БД

Для проєктування бази даних можна використовувати Schema View у відповідних засобах роботи з СУБД (Microsoft Sql Server Management Studio, PL/SQL Developer та інші) або вбудований засіб Microsoft Office Access.

Для створення таблиць необхідно натиснути кнопку «Створити» (CREATE).

Це представлено на рисунку 2.15.

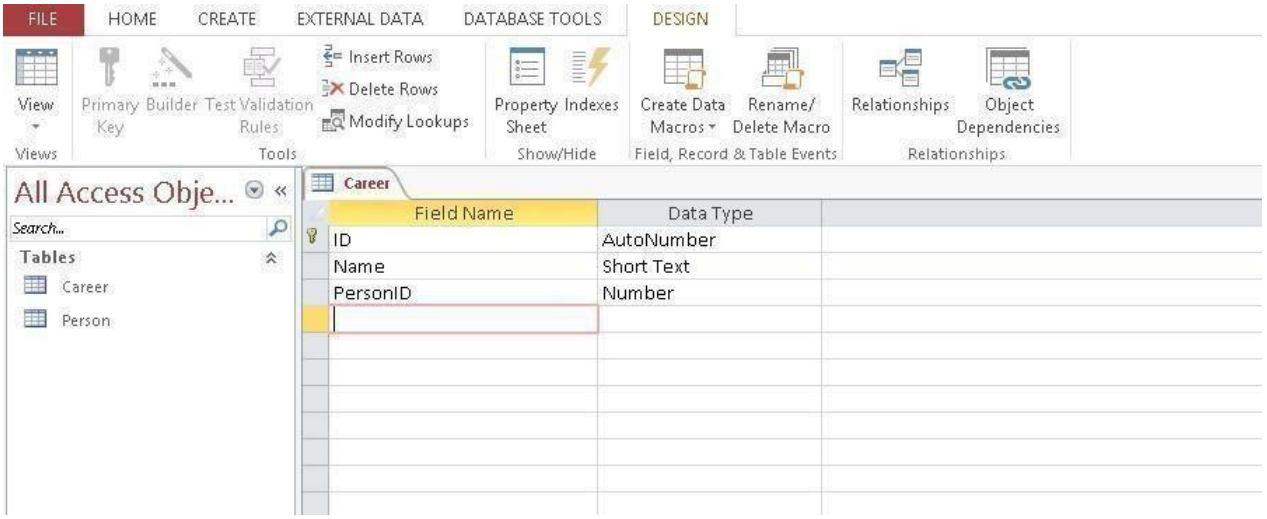


Рисунок 2.15. Створення таблиці в MS Access

Після заповнення полів та їх типів необхідно призначити первинний ключ. Після цього можна виставитизв'язок між таблицями у вікні «Зв'язки»(Relationships). Зв’язко представлено на рисунку 2.16.

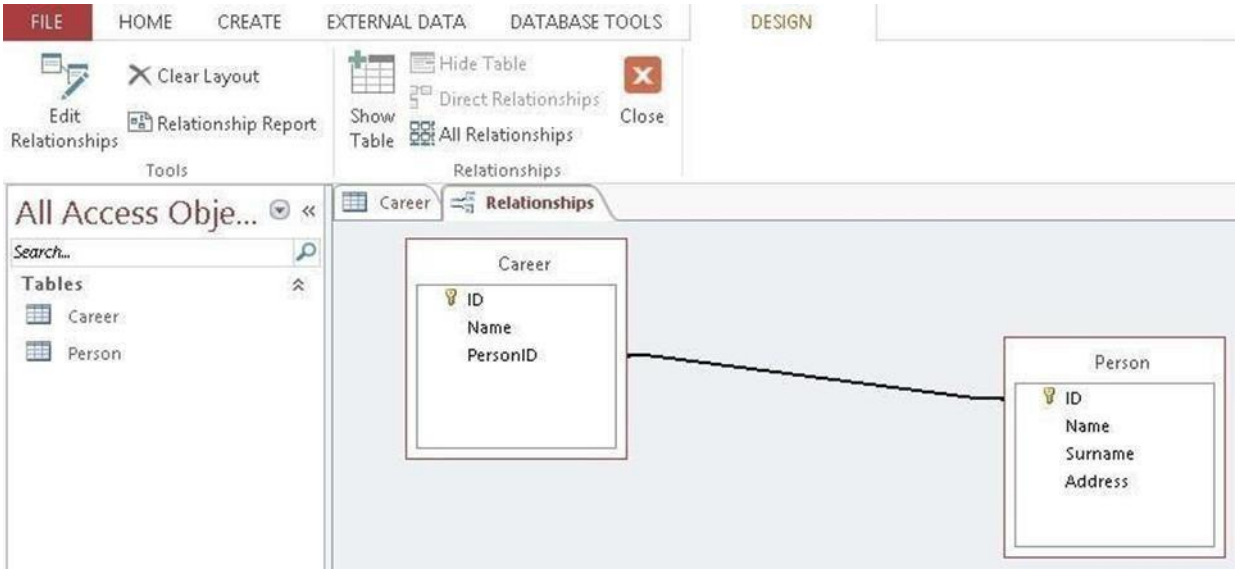


Рисунок 2.16. Зв’язок між двома таблицями на діаграмі в MS Access

Хід роботи

1. Розробка діаграми варіантів використання (Use Case Diagram)

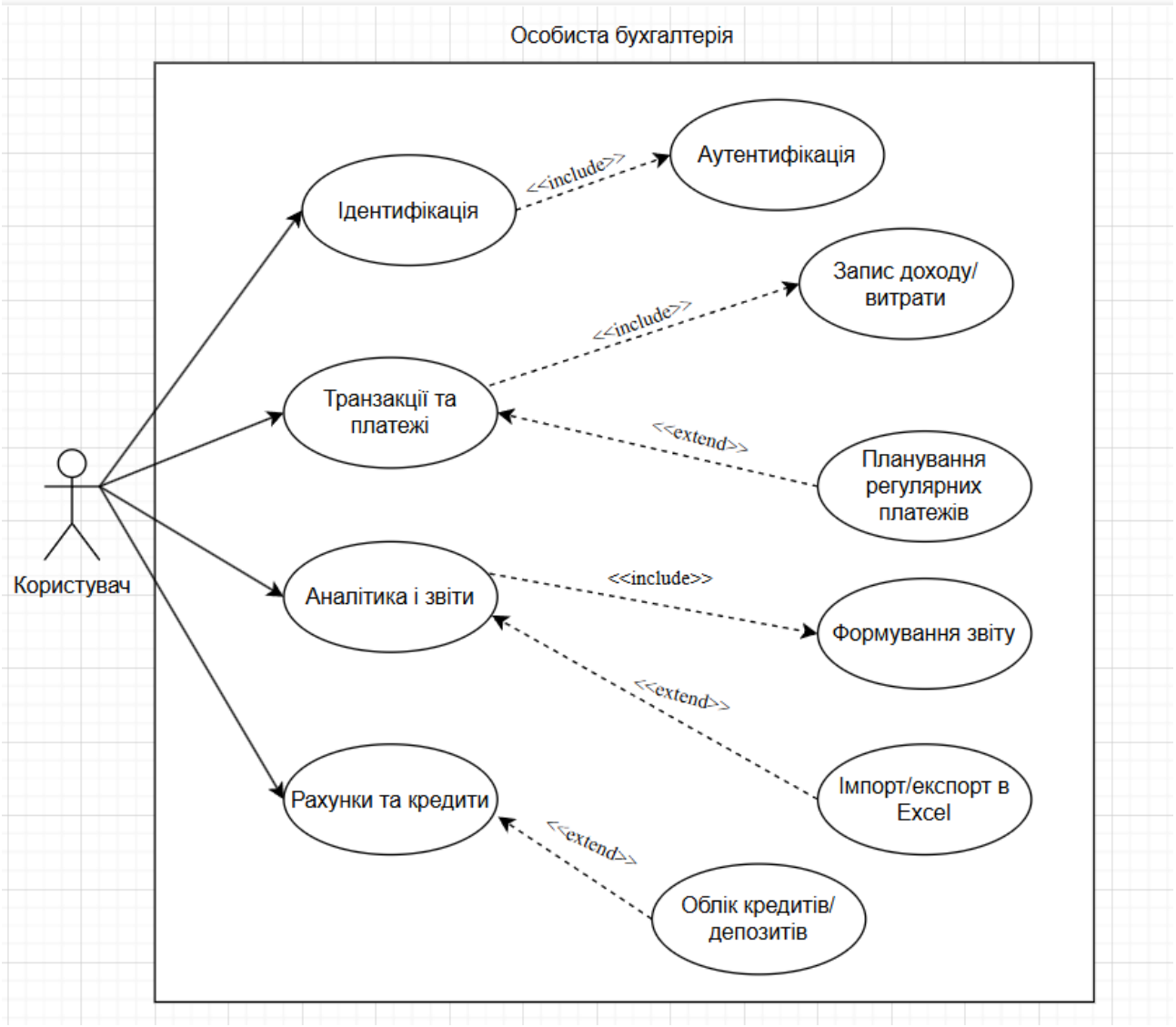


Рисунок 1 – Use case діаграма для звичайного користувача

2. Розробка діаграми класів предметної області

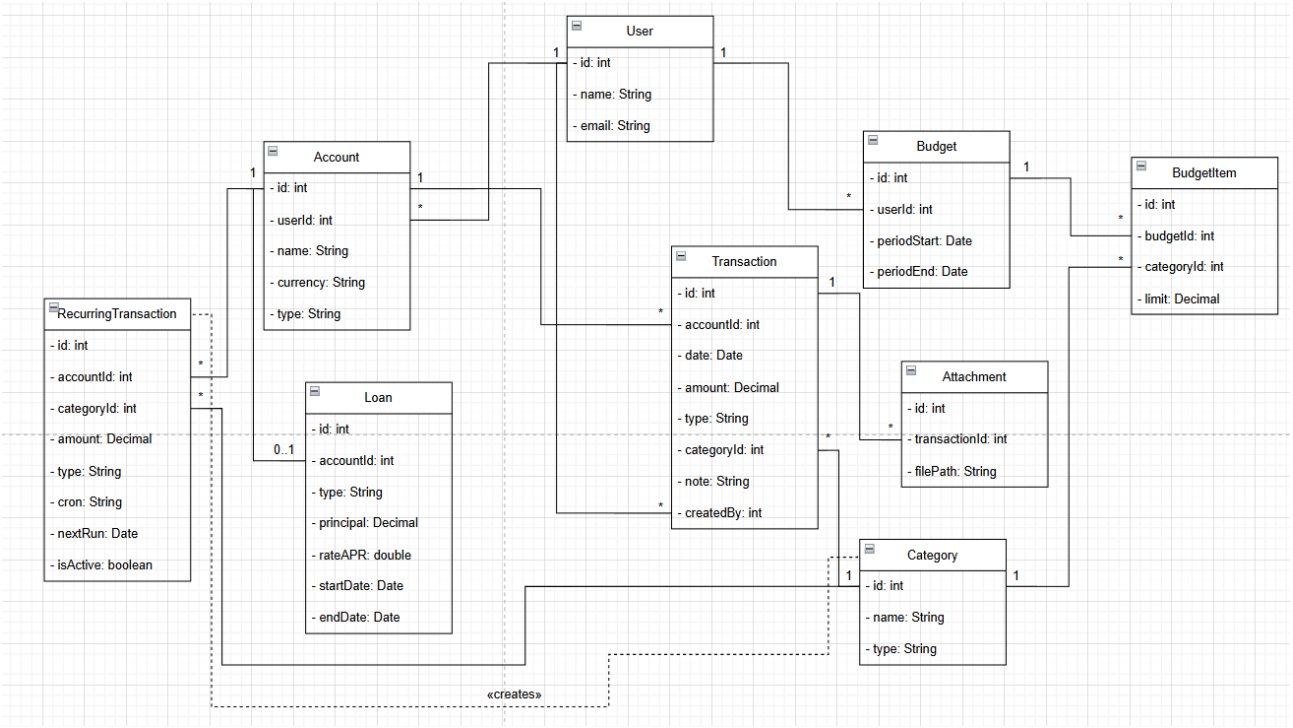


Рисунок 2 –Діаграма класів

3 варіанти сценарії використання

Сценарій 1: Запис доходу/витрати

Передумови:

- Користувач автентифікований у системі
- Існує хоча б один рахунок і одна категорія транзакцій

Постумови:

- У системі створено нову транзакцію.
- Баланс рахунку оновлено

Взаємодіючі сторони:

Користувач, Система.

Короткий опис:

Цей варіант використання описує процес додавання доходу або витрати користувачем у систему.

Основний потік подій:

1. Користувач обирає функцію «Додати транзакцію»
2. Система пропонує ввести дані: рахунок, категорію, дату, суму, тип (дохід/витрата), примітку
3. Користувач вводить усі необхідні дані
4. Система створює нову транзакцію, оновлює баланс рахунку та повідомляє про успішне збереження

Винятки:

- Якщо користувач не вказав категорію то система просить вибрати або створити нову
- Якщо валюта рахунку не збігається з введеною транзакцією, то система виконує конвертацію або видає повідомлення про помилку

Примітки:

Можливе додавання вкладення (скан чеку).

Сценарій 2: Планування регулярних платежів

Передумови:

- Користувач увійшов у систему
- Існує хоча б один рахунок і категорія

Постумови:

- У системі збережено нове правило періодичної транзакції
- У зазначений час будуть автоматично створюватися відповідні транзакції

Взаємодіючі сторони:

Користувач, Система.

Короткий опис:

Цей варіант описує створення періодичного платежу (наприклад, щомісячної оплати оренди).

Основний потік подій:

1. Користувач обирає функцію «Створити періодичну транзакцію»
2. Система пропонує ввести дані: рахунок, категорію, суму, періодичність, дату початку
3. Користувач вводить дані й підтверджує створення
4. Система зберігає правило і планує першу дату виконання

Винятки:

- Якщо користувач вводить некоректну періодичність то система видає повідомлення і просить повторити введення
- Якщо на рахунку немає достатньо коштів у момент виконання то система позначає транзакцію як «очікує підтвердження»

Примітки:

Для регулярних доходів (наприклад, зарплата) процес аналогічний.

Сценарій 3: Експорт даних у Excel

Передумови:

- Користувач увійшов у систему.
- У базі є хоча б одна транзакція.

Постумови:

- Згенеровано файл Excel із даними про транзакції.

Взаємодіючі сторони:

Користувач, Система.

Короткий опис:

Цей варіант описує процес експорту фінансових даних у формат Excel для подальшого аналізу.

Основний потік подій:

1. Користувач обирає функцію «Експортувати дані»
2. Система пропонує вибрати період (наприклад, за місяць)
3. Користувач підтверджує вибір
4. Система формує Excel-файл та надає можливість його завантажити

Винятки:

Якщо за вибраний період немає даних → система показує повідомлення «Немає даних для експорту»

Примітки:

Експорт може виконуватися і у формат CSV

Основні класи та структура бази даних системи:

Схема данных | Categories | Loans | RecurringTransaction

Имя поля	Тип данных
ID	Счетчик
Name	Короткий текст
Email	Короткий текст

Сервис | Показать или скрыть

Схема данных | Categories | Loans | RecurringTransactior

Имя поля	Тип данных
ID	Счетчик
UserID	Числовой
PeriodStart	Дата и время
PeriodEnd	Дата и время

Схема данных | Categories | Loans | RecurringTransacti

Имя поля	Тип данных
ID	Счетчик
BudgetID	Числовой
CategoryID	Числовой
Limit	Денежный

Сервис | Показать или скрыть | События полей, записей и таблиц

Transactions | RecurringTransactions

Имя поля	Тип данных
ID	Счетчик
AccountID	Числовой
CategoryID	Числовой
Amount	Денежный
Type	Короткий текст
Cron	Короткий текст
NextRun	Дата и время
IsActive	Логический

Loans

Имя поля	Тип данных
ID	Счетчик
AccountID	Числовой
Type	Короткий текст
Principal	Денежный
RateAPR	Числовой
StartDate	Дата и время
EndDate	Дата и время

Transactions | RecurringTransactions

Имя поля	Тип данных
ID	Счетчик
AccountID	Числовой
Date	Дата и время
Amount	Денежный
Type	Короткий текст
CategoryID	Числовой
Note	Короткий текст
CreatedBy	Числовой

Сервис | Показать или скрыть | События полей, запи

Loans | Categories

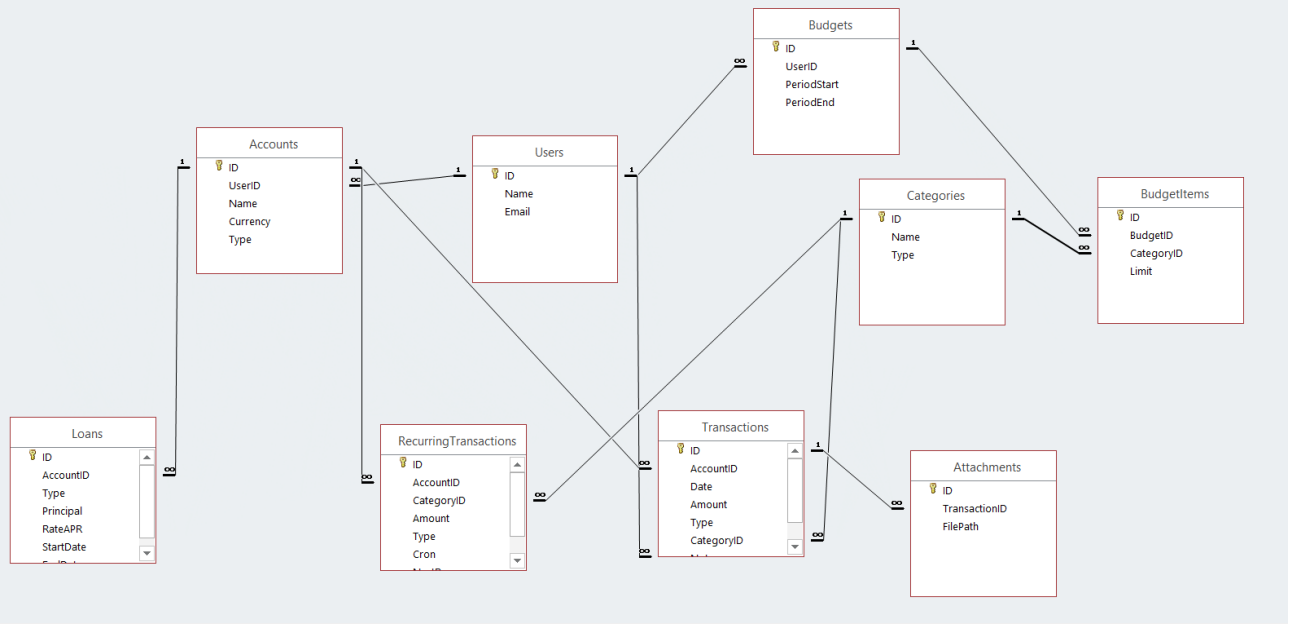
Имя поля	Тип данных
ID	Счетчик
Name	Короткий текст
Type	Короткий текст

Budgets

Имя поля	Тип данных
ID	Счетчик
UserID	Числовой
PeriodStart	Дата и время
PeriodEnd	Дата и время

BudgetItems | Attachments

Имя поля	Тип данных
ID	Счетчик
TransactionID	Числовой
FilePath	Короткий текст



Діаграма класів реалізованої частини системи:

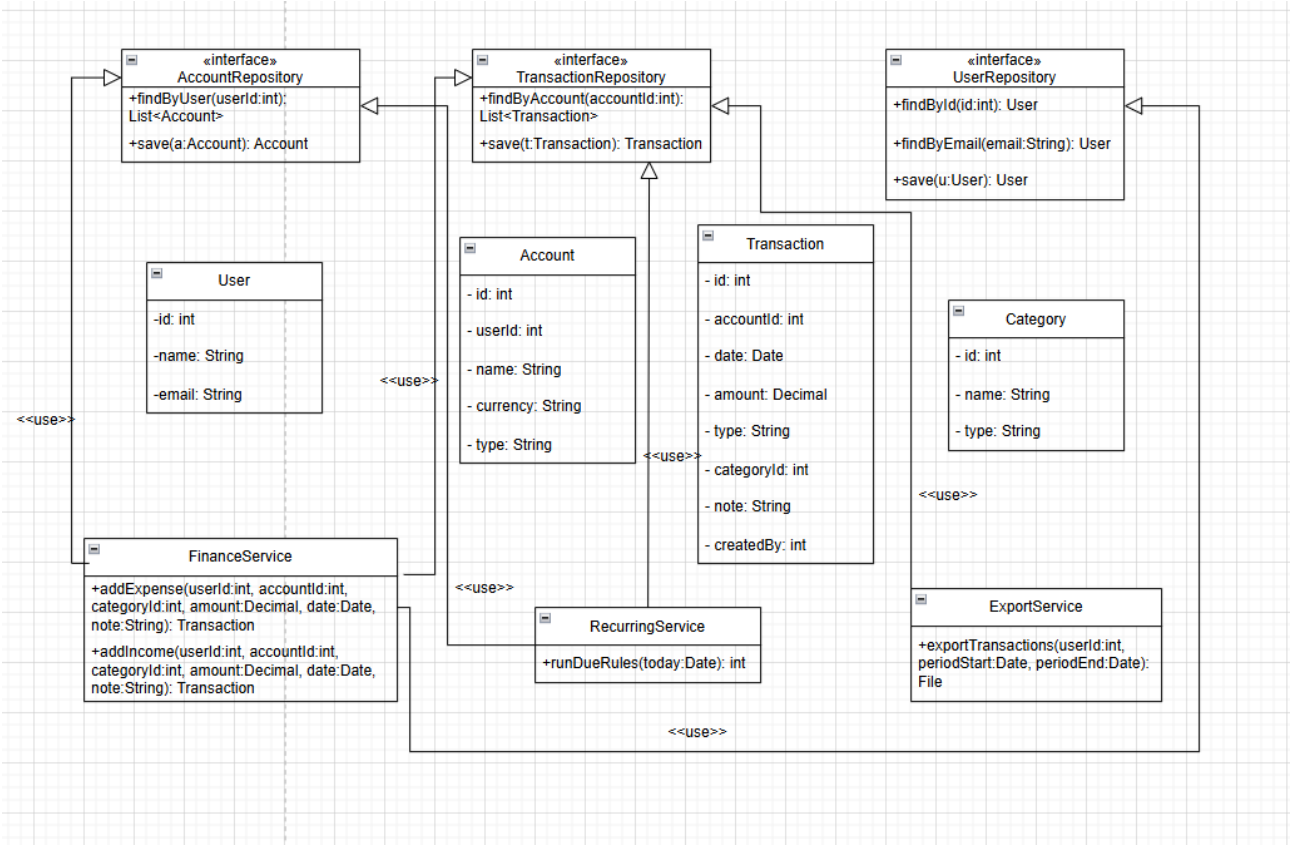


Рисунок 3 – Діаграма класів реалізованої частини системи

На діаграмі наведено структуру реалізації системи «Особиста бухгалтерія». Для доступу до даних використано шаблон Repository, що дозволяє відокремити бізнес-логіку від роботи з базою даних.

- Сутності (Entity-класи): User, Account, Transaction, Category. Вони відображають основні об’єкти предметної області та містять атрибути для зберігання інформації.
- Інтерфейси репозиторіїв: UserRepository, AccountRepository, TransactionRepository. Визначають методи доступу до даних (пошук, збереження, отримання списків).

Сервіси (Service-класи):

- FinanceService відповідає за додавання доходів та витрат
- RecurringService реалізує роботу з періодичними транзакціями

- ExportService виконує експорт транзакцій за вибраний період

Сервіси звертаються до репозиторіїв через залежності (Dependency), що дозволяє легко змінювати або підмінювати реалізації без змін у бізнес-логіці.

Таким чином, діаграма демонструє взаємодію між сутностями, інтерфейсами доступу до даних та сервісами, які реалізують бізнес-функціональність системи.

Висновок: у ході роботи я ознайомилась з основами проектування програмних систем. Було розроблено діаграму варіантів використання, діаграму класів предметної області та реалізованої частини системи «Особиста бухгалтерія». На основі побудованої моделі створено структуру бази даних у MS Access та встановлено зв’язки між таблицями. Отримані результати демонструють вміння застосовувати UML та принципи проектування для організації даних і бізнес-логіки системи.